BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Trương Hoàng Đăng Khôi
MSSV / Lớp	3124720035/ DNT1241

Mục tiêu học tập

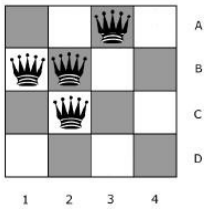
- Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).
- Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.
- Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.
- Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.
- Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Di chuyển quân hậu từ C3 đến D4 (hoặc di chuyển hậu từ A3 đến A4):
Vị trí các hậu mới: B1, B2, A3, D4
Số cặp xung đột mới = 2 (Cặp B1-B2 cùng hàng B, B2-A3 cùng đường chéo).
Kết quả sau vòng lặp 1:
Cột 1: Quân hậu ở B1 (không đổi)
Cột 2: Quân hậu ở B2 (không đổi)
Cột 3: Quân hậu ở A3 (không đổi)
Cột 4: Quân hậu ở D4 (đã di chuyển từ C3 sang D4)

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☐ B) Cực trị cục bộ (local optimum)

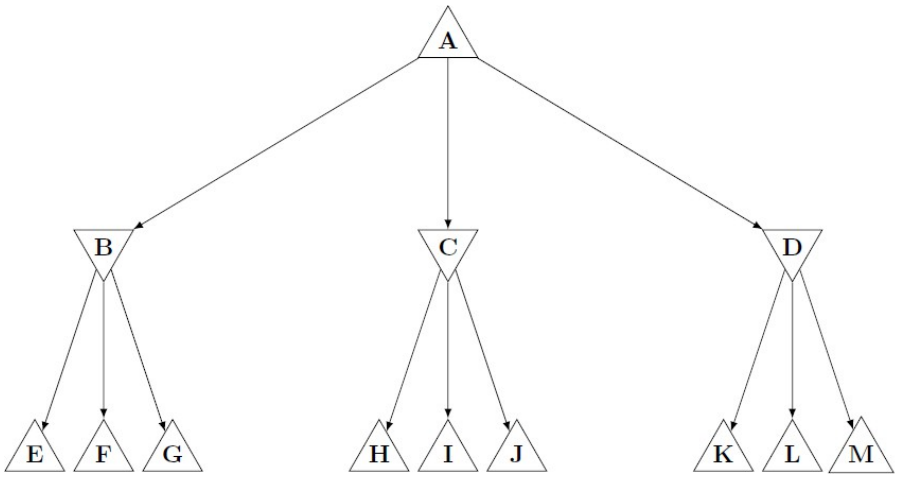
☒ C) Cực trị toàn cục (global optimum)

☐ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☒ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Xét cây con D (MIN):
- Xét K=23 =>D = 23 (Beta $_D$ = 23).
- Xét L=20 => D = 20 (Beta $_D$ = 20).
- Xét M=26 => D = 20.

Giá trị của D = 20. Nút A (MAX) cập nhật Alpha $_A$ = 20.

Xét cây con C (MIN):
- Xét H=17. Giá trị tạm thời của C = 17 (Beta $_C$ = 17).
- Vì Beta $_C$ = 17 <= Alpha $_A$ = 20, xảy ra cắt tỉa alpha tại nút C.
- Không duyệt các nút con còn lại của C: I, J

Xét cây con B (MIN):
- Xét E=5. Giá trị tạm thời của B = 5 (Beta $_B$ = 5).
- Vì Beta $_B$ = 5 <= Alpha $_A$ = 20, xảy ra cắt tỉa alpha tại nút B.
- Không duyệt các nút con còn lại của B: F, G.

Các nút bị cắt tỉa: I, J, F, G

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

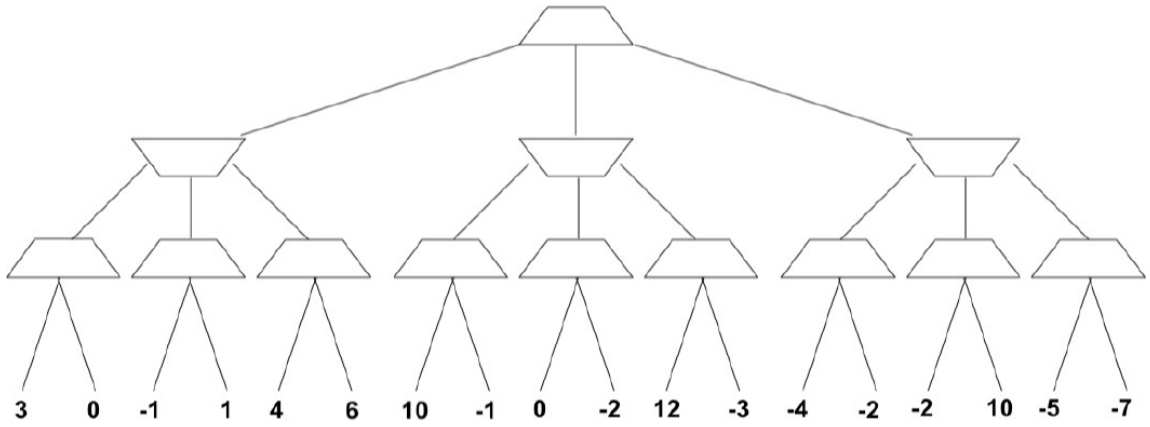
☒ Có

☐ Không

Sắp xếp thứ tự duyệt nút trong Iterative Deepening làm thay đổi thứ tự thăm các nhánh để tăng hiệu quả cắt tỉa, không làm thay đổi cấu trúc cây hay giá trị thực tế của các nút lá. Do đó, Alpha-Beta vẫn đảm bảo tìm ra kết quả chính xác giống như thuật toán Minimax đầy đủ

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

- Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.
- Ghi giá trị được trả về tại mỗi nút trung gian.
- Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.
- Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

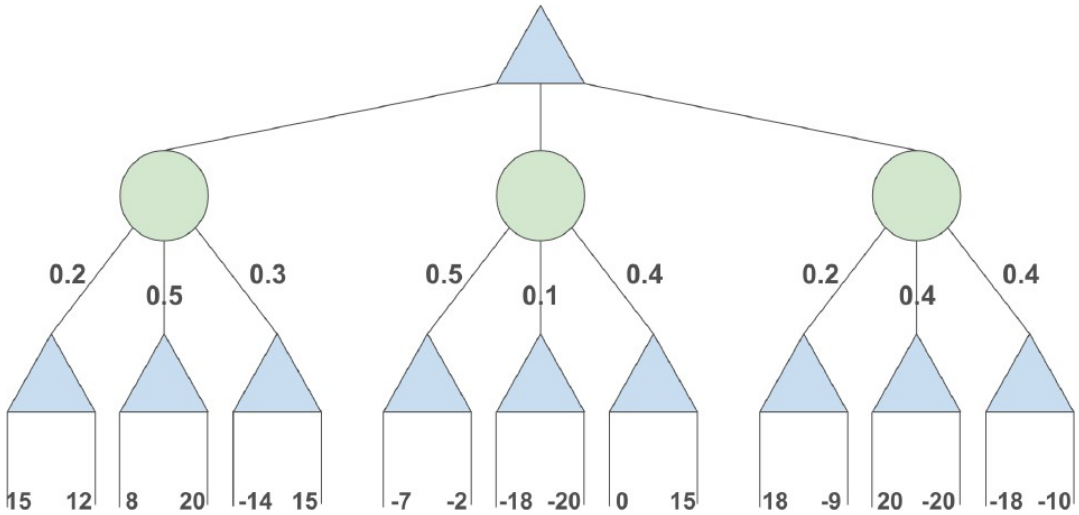
Giá trị trả về tại các nút trung gian:
- v (C_1) = 3, v (C_2) = 1, v (C_3) = 4
- v (C_4) = 10, v (C_5) = 0
- v (C_7) = -2
- v(B_1) = 1, v(B_2) = 0, v(B_3) = -2

Các nhánh/lá bị cắt tỉa (đánh dấu X):
- Lá 6 (thuộc C_3).
- Cây con C_6(lá 12, -3).
- Cây con C_8 (lá -2, 10) và C_9 (lá -5, -7).

Giá trị Minimax tại gốc A: 1.
Nước đi tối ưu của MAX: B_1 (nhánh ngoài cùng bên trái).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Giá trị tại các nút MAX tầng dưới :

- Nhóm 1 (trái): max(15, 12) = 15 ; max(8, 20) = 20 ; max(-14, 15) = 15
- Nhóm 2 (giữa): max(-7, -2) = -2 ; max(-18, -20) = -18 ; max(0, 15) = 15
- Nhóm 3 (phải): max(18, -9) = 18 ; max(20, -20) = 20 ; max(-18, -10) = -10

Giá trị tại các nút Chance:

- Nút Chance 1 (trái) = (0.2 * 15) + (0.5 * 20) + (0.3 * 15) = 3 + 10 + 4.5 = 17.5
- Nút Chance 2 (giữa) = (0.5 * -2) + (0.1 * -18) + (0.4 * 15) = -1 - 1.8 + 6 = 3.2
- Nút Chance 3 (phải) = (0.2 * 18) + (0.4 * 20) + (0.4 * -10) = 3.6 + 8 - 4 = 7.6

Giá trị tại nút gốc MAX:

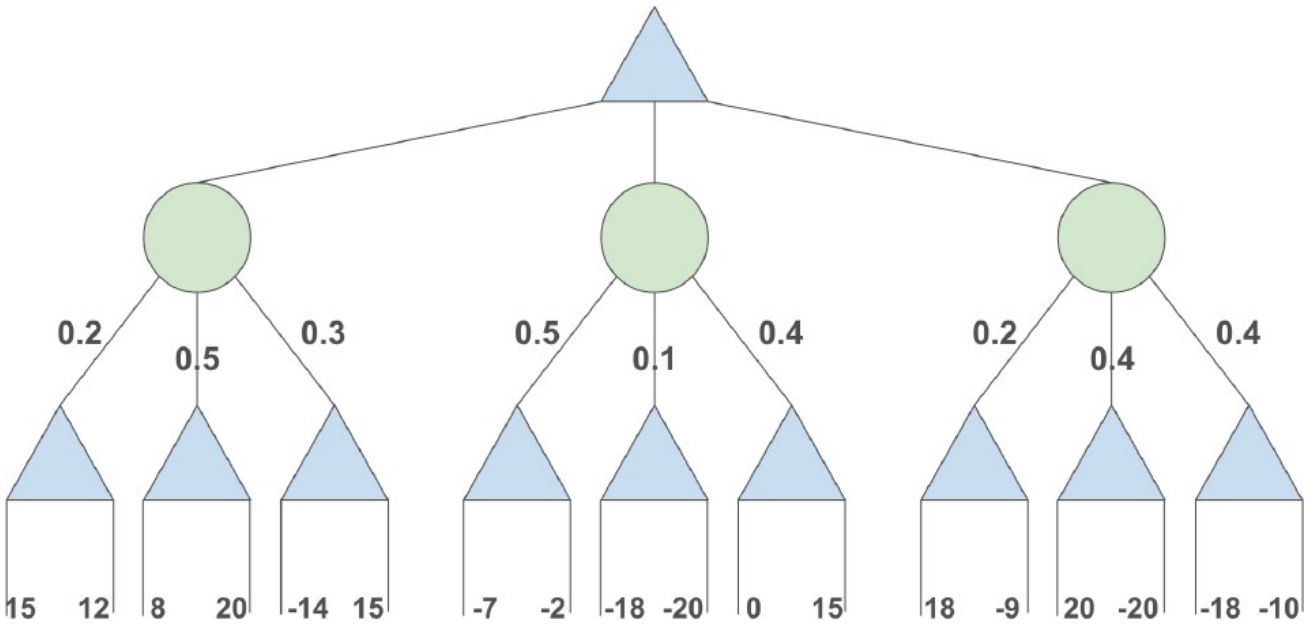
max(17.5, 3.2, 7.6) = 17.5

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không thể cắt tỉa nút nào vì:

Do bài toán không cho giới hạn cận trên/cận dưới đối với giá trị ở các nút lá, bất kỳ một lá nào chưa được duyệt vẫn có thể mang giá trị cực lớn làm thay đổi hoàn bộ giá trị kỳ vọng của nút Chance đó. Vì vậy, thuật toán Expectimax bắt buộc phải duyệt toàn bộ cây.

c/ Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4©

c) Cắt tỉa các nhánh không cần thiết khi các nút lá thuộc [-20, 20]:

1. Nút Chance 1 (Trái):
   - Duyệt toàn bộ, tính được giá trị = 17.5
   - Đặt giá trị ngưỡng tại gốc MAX: alpha = 17.5

2. Nút Chance 2 (Giữa):
   - Duyệt nhánh con 1 có giá trị = -2 (trọng số 0.5)
   - Tối đa hóa giá trị kỳ vọng còn lại với U_max = 20:
     Upper Bound = (0.5 * -2) + (0.1 * 20) + (0.4 * 20) = 9.0
   - Vì Upper Bound (9.0) < alpha (17.5) -> Cắt tỉa toàn bộ 2 nhánh còn lại của Chance 2 (các nhánh chứa lá -18, -20 và 0, 15).

3. Nút Chance 3 (Phải):

- Duyệt nhánh con 1 = 18 -> Upper Bound = 19.6 > 17.5 (chưa cắt)
- Duyệt nhánh con 2 = 20 -> Upper Bound = 19.6 > 17.5 (chưa cắt)
- Duyệt nhánh con 3 = -10 -> Giá trị thực tế của Chance 3 = 7.6

* Các nhánh/nút bị cắt tỉa:
- Nút MAX chứa cặp lá [-18, -20]
- Nút MAX chứa cặp lá [0, 15]

* Giá trị tại nút gốc MAX: max(17.5, 9.0_bounded, 7.6) = 17.5

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Phạm vi lan truyền ràng buộc:
- Forward Checking: Chỉ lan truyền cục bộ từ biến vừa gán sang các biến chưa gán có liên kết trực tiếp (1 bước).
- AC-3: Lan truyền toàn cục trên toàn mạng lưới ràng buộc cho đến khi đạt trạng thái ổn định.

Thời điểm thực hiện:
- Forward Checking: Chỉ chạy sau mỗi bước gán giá trị cho một biến.
- AC-3: Chạy ở bước tiền xử lý hoặc sau mỗi bước gán giá trị trong quá trình tìm kiếm (trong thuật toán MAC).

Chi phí tính toán:
Forward Checking: Chi phí thấp, thực hiện nhanh ở từng bước.
AC-3: Chi phí tính toán cao hơn, nhưng giúp phát hiện mâu thuẫn sớm hơn và thu hẹp không gian tìm kiếm tốt hơn.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

Theo nguyên lý "Fail-first" . Chọn biến có ít giá trị hợp lệ còn lại nhất. Nếu nhánh này dẫn đến đường cùng, thuật toán sẽ quay lui ngay lập tức mà không lãng phí thời gian duyệt sâu.

ii) Least Constraining Value (LCV)

Theo nguyên lý "Succeed-first" . Chọn giá trị ít làm giảm sự lựa chọn của các biến hàng xóm chưa gán nhất, làm tăng tối đa khả năng tìm thấy lời giải ngay ở nhánh đầu tiên.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.
- Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C.
- Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Bổ sung hai ràng buộc sau:

- A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
- D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

Biến: X = {A, B, C, D, E} (mỗi biến đại diện cho một máy bay)
Miền giá trị: D_i= {1, 2, 3, 4} , {Quốc tế, Nội địa}} với mọi I thuộc {A, B, C, D, E}.
Ràng buộc đơn nguyên:
B time = 1
D time >= 3 (D time thuộc {3, 4})
A time <= 2 (A time thuộc {1, 2})
A runway= B runway = C runway = Quốc tế
D runway = E rungway = Nội địa
Ràng buộc nhị phân:
D time < C time
(A time != B time) hoặc (Arungway != B rungway)

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Miền giá trị mới: DA = DB = DC = DD = DE = {1, 2, 3, 4}.
Đồ thị ràng buộc:
Cụm Quốc tế: Cung nối giữa các cặp (A, B), (B, C), (A, C) [ràng buộc khác khung giờ].
Cụm Nội địa: Cung nối giữa cặp (D, E) [ràng buộc khác khung giờ].
Cung liên cụm: Cung nối giữa (D, C) [ràng buộc D_time < C_time].

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Sau khi áp dụng các ràng buộc đơn nguyên ban đầu:
.
- DA = {1, 2}
- DB = {1}
- DC = {1, 2, 3, 4}
- DD = {3, 4}
- DE = {1, 2, 3, 4}

Sau khi lọc Arc Consistency (xét B = 1 => A != 1, và D < C => C > 3):
.
- DA = {2}
- DB = {1}
- DC = {4}
- DD = {3}
- DE = {1, 2, 4}

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo

đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

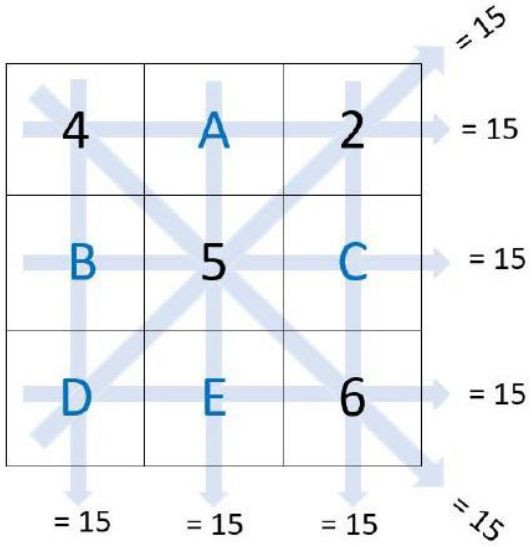
A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

1. Khởi tạo sau Unary: DA = {1, 2}, DB = {1}, DC = {1, 2, 3, 4}, DD = {3, 4}, DE = {1, 2, 3, 4}.
2. Chọn B (MRV, |DB| = 1): Gán B = 1 -> Forward Checking cập nhật DA = {2}, DC = {2, 3, 4}.
3. Chọn A (MRV, |DA| = 1): Gán A = 2 -> Forward Checking cập nhật DC = {3, 4}.
4. Chọn D (MRV, |DD| = 2, hòa chọn D theo chữ cái): Gán D = 3 -> Forward Checking cập nhật DC = {4}, DE = {1, 2, 4}.
5. Chọn C (MRV, |DC| = 1): Gán C = 4.
6. Chọn E (MRV, |DE| = 3): Gán E = 1.

Thứ tự gán và phân công hoàn chỉnh:
(B, 1), (A, 2), (D, 3), (C, 4), (E, 1)

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

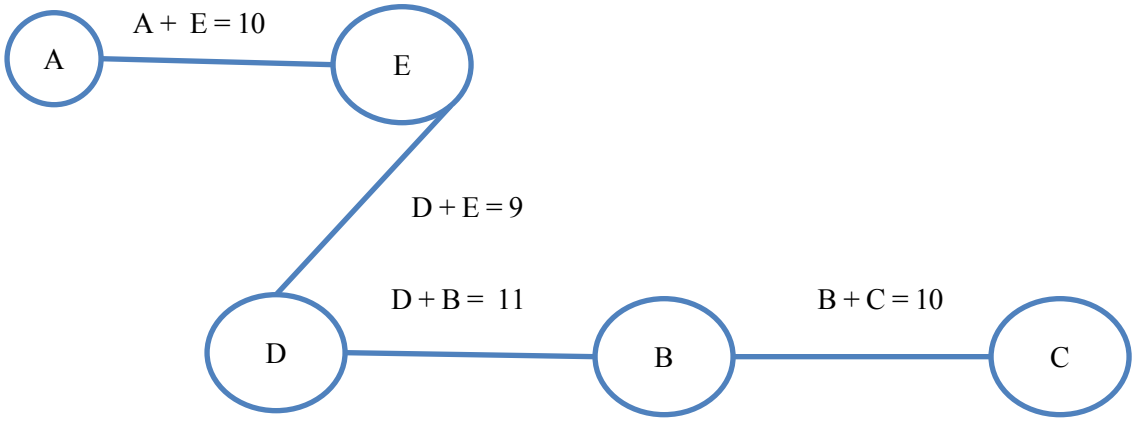
a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Tập biến: X = {A, B, C, D, E} .
Miền giá trị ban đầu : {1, 3, 7, 8, 9}.
DA = DB = DC = DD = DE = {1, 3, 7, 8, 9}
Các ràng buộc:

Ràng buộc khác nhau từng đôi một : (A, B, C, D, E).
Ràng buộc tổng theo hàng, cột, đường chéo (bằng 15):

1. Hàng 1: 4 + A + 2 = 15 => A = 9
2. Hàng 2: B + 5 + C = 15 => B + C = 10
3. Hàng 3: D + E + 6 = 15 => D + E = 9
4. Cột 1: 4 + B + D = 15 => B + D = 11
5. Cột 2: A + 5 + E = 15 => A + E = 10
6. Cột 3: 2 + C + 6 = 15 => C = 7
7. Đường chéo phụ: D + 5 + 2 = 15 => D = 8

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.



c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Kết quả sau khi lọc Arc Consistency (AC-3):

- Từ phương trình đơn nguyên thu được: A = 9 và C = 7.
- Do D + 5 + 2 = 15 => D = 8.
- Thay D = 8 vào B + D = 11 => B = 3.
- Thay D = 8 vào D + E = 9 => E = 1.

Một nghiệm cho Magic Square:

A = 9; B = 3; C = 7; D = 8; E = 1

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

- AJ = {Đỏ, Vàng, Xanh lá}
- Sunny = {Đỏ, Vàng, Hồng}
- Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Số lần backtrack: 0 lần
Thứ tự gán:

- Ellyse = Hồng (MRV chọn Ellyse do miền nhỏ nhất = 2; LCV chọn Hồng vì chỉ loại 1 màu của Sunny).
- Sunny = Đỏ (MRV chọn Sunny do miền nhỏ hơn AJ; LCV hòa chọn Đỏ theo thứ tự cầu vồng).
- AJ = Vàng (Gán giá trị còn lại theo thứ tự cầu vồng).

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Số lần backtrack: 1 lần
AJ = Đỏ (miền lớn nhất, gây loại nhiều màu nhất).
Sunny = Hồng => Dẫn đến Ellyse bị rỗng miền giá trị..
Thuật toán buộc phải backtrack 1 lần để hủy gán Sunny = Hồng.